



Software build automation tools

Lab – Module 2

- Autor: dr inż. Aleksander Rewak
- Wrocław, 2026

Theoretical Introduction: GitHub as a Computing Platform



In modern software development, GitHub serves two primary roles:

1. **Version Control System (Git):** It stores your code and the history of all changes.
2. **Runtime Environment (GitHub Actions):** It provides on-demand computing power (servers) to "bring your code to life" for testing and deployment.

The "Runner" Mechanism (Virtual Machines)

When a configuration file is detected in your repository, GitHub triggers a **Runner**. A Runner is a temporary **Virtual Machine (VM)** usually running Linux (Ubuntu).

1. **Isolation:** The environment is "sterile." Every session starts from scratch with a clean OS.
2. **Objective Validation:** It verifies that the code works in a standard server environment, eliminating the "it works on my machine" excuse.

Scenarios to experiment



1. Project Initiation & Local Setup (Role A): Setting up the master repository on GitHub and creating the initial `geometry.py` source code, `test_logic.py` test suite, and `.gitignore` file locally.
2. Collaborative Forking & Branching (Role B): Role B creates a personal copy (Fork) of the project and works in an isolated `feature-circle-area` branch to maintain production stability.
3. The Development Bridge (Pull Request): Role B proposes a new `circle_area` function to Role A. Role A then performs a Code Review, providing feedback or requesting changes before the code is merged.
4. The "Management" Flow (Issue Tracking): Demonstrates linking a specific task (Issue) to a Pull Request using "Magic Keywords" (e.g., `Closes #1`) so the task automatically closes when the code is merged.
5. Resolving Merge Conflicts: A simulation of a "real-world" headache where both roles modify the same line of code, requiring a manual resolution in VS Code to decide which version to keep.
6. Continuous Integration (CI) Experiment: Configuring a GitHub Actions pipeline (`main.yml`) to automatically boot a Linux Virtual Machine and validate the code through automated tests upon every push.
7. The "Human Error" Test (Simulated Failure): Intentionally breaking a mathematical formula to witness how the automated system detects the error, turns "Red," and prevents unstable code from being merged.

GitHub project - Engineering Geometry Library



The "**Engineering Geometry Library**" project simulation.

This guide is designed to play both roles (Lead and Contributor) to demonstrate a professional industry standard.

Role	Account / Browser	Responsibility
Role A (Lead)	roleA@mail(Chrome)	Owns the Main Repo (MeritoRewak/SBA).
Role B (Junior)	roleB@mail (Firefox/Incognito)	Owns the Fork (a personal copy of the code).

Step 1: Project Initiation (Role A - Lead)



1. **GitHub:** Create a public repository named *SBA*. (*DO NOT* check "Add a README file", "Add .gitignore", "Choose a license")
2. *Create your project in VS Code*
 - create folder on your disk and open it
 - create python script (next slides)

Step 2: Create the Source Code (geometry.py)



First, we need to create the "product"—the actual code that performs calculations.

- Create a file named geometry.py.
- Add the following mathematical functions:

```
import math
def triangle_area(base, height):
        """Calculates the area of a triangle."""
        return 0.5 * base * height
def square_area(side):
        """Calculates the area of a square."""
        return side ** 2

if __name__ == "__main__":
        # Demonstration of current logic
        print(f"Triangle Area (b=10, h=5): {triangle_area(10, 5)}")
        print(f"Square Area (a=4): {square_area(4)}")
```

Step 3: Adding the .gitignore



Since you didn't select it on GitHub, you should create it now in VS Code to keep your repository clean.

Create a new file in your root folder named .gitignore.

Add these lines to prevent Python's temporary files from being uploaded:

```
# Python bytecode
```

```
__pycache__/
```

```
*.py[cod]
```

```
# Environments
```

```
.venv/
```

```
env/
```

```
venv/
```

```
# VS Code specific
```

```
.vscode/
```

Step 4a: Connecting to GitHub - *Using the VS Code Interface (Fastest)*



VS Code has built-in integration that handles the "handshake" with GitHub automatically.

- 1. Identity Check - open the Terminal and type: `git config user.name` and `git config user.email` to check the current ID. If it is not yours, type: `git config --global user.name „Your Name”`, `git config --global user.email „your@email.com”`*
- 2. Initialize Git: Click on the Source Control icon on the left sidebar (looks like a branch/node) and click the Initialize Repository button.*
- 3. Stage & Commit: * Type a message in the box (e.g., "Initial commit"). Click Commit (or the checkmark icon).*
- 4. Publish to GitHub:*
 - Click the blue Publish Branch button.
 - Select Publish to GitHub public repository.
 - VS Code will ask for permission to sign in to GitHub. Follow the browser prompts.
 - Once authorized, VS Code creates the repo for you and pushes the code.

Step 4b: Connecting to GitHub – VS Code Terminal



1. Initialize the local folder
`git init`

2. Add all your files (geometry.py, test_logic.py, .gitignore)
`git add .`

3. Create the first commit
`git commit -m "Initial project setup"`

4. Link your local project to the GitHub URL you copied
`git remote add origin`
`https://github.com/MeritoRewak/SBA.git`

5. Rename the default branch to 'main'
`git branch -M main`

6. Push the code (the -u flag remembers the link for next time)
`git push -u origin main`

Identity Check - open the Terminal and type: `git config user.name` and `git config user.email` to check the current ID. If it is not yours, type: `git config --global user.name „Your Name”`, `git config --global user.email „your@email.com”`

Step 5: Forking & Branching (Role B – The Contributor)



Goal: Get a safe copy to work on without breaking the Lead's code.

1. On GitHub (Firefox): Log in as Role B. Go to <https://github.com/RoleA/SBA>.
2. The Fork: Click the Fork button (top right). You now have your own copy
3. In VS Code (Folder 2: Geometry_RoleB):
 - Open a new window and Sign In as RoleB - Contributor
 - Clone your fork:

```
git clone https://github.com/RoleB/SBA  
cd SBA
```
4. The Branch: Create a dedicated "work zone" for a new feature:

```
git checkout -b feature-circle-area
```

(Explanation: We never work directly on 'main' to keep the production code stable).

Step 6.1: The Bridge (Role B → Role A)



Goal: Propose the new feature to the Lead.

1. Code (Role B): In the „RoleB" VS Code, add the circle_area function to geometry.py.

```
def circle_area(radius):  
    """Calculates the area of a circle."""  
    return math.pi * (radius ** 2)
```

2. Push (Role B): Send the changes to your private fork:

```
git add geometry.py  
git commit -m "Add circle area formula"  
git push origin feature-circle-area
```

Step 6.2: The Bridge (Role B → Role A)



3. Pull Request (Role B):

- Go to your fork on GitHub.
- Click the yellow "Compare & pull request" button.
- Check direction (The Arrow):
 - base repository: RoleA/SBA (destination) base: main
 - head repository: RoleB/SBA (source) compare: feature-circle-area
- Write a short description: "Added circle area calculation using the math module."
- Click Create Pull Request.

The Pull Request is the 'Gatekeeper.' It ensures that the Lead (Role A) is always in control of the project's health, while allowing the Contributor (Role B) the freedom to experiment safely in their own Fork. Why is essential:

- **Safety Gate:** It creates a "quarantine" for the Lead (Role A) to review the code for bugs before it touches the main project.
- **Conflict Prevention:** GitHub automatically compares both versions to ensure new changes won't accidentally overwrite or crash the existing code.

Step 7: Code Review (Role A)



Goal: Inspect the contributor's work before merging it into the main project.

1. **On GitHub (Chrome):** Log in as **Role A (Merito)**. Go to your **SBA** repository.
2. **Access the Request:** Click the **Pull Requests** tab. Click on the specific request sent by **Role B** (e.g., *"Added circle area calculation"*).
3. **The Review:** Look for the sub-tabs near the top of the page. Click on the **"Files changed"** tab (located to the right of "Conversation" and "Commits").
4. **Feedback:** * To comment on a specific line: Hover over the line of code and click the blue **[+]** icon.
 - Type your comment (e.g., „All OK” or „Please add a docstring explaining the radius parameter”) and click Start a review.
5. **The Verdict:** Click the green **Finish your review** button (top right). Select **"Comment"**, **"Approve"**, or **"Request changes"**, then submit.
6. Merge Pull request

Step 8. Resolving a Merge Conflict (The "Real World" Headache)



This happens in every real job.

1. The Setup:

- Role A modifies geometry.py directly on the main branch (e.g., adds a comment at the top).
- Role B modifies the same line in their feature branch.

2. The Action: When Role B tries to merge, GitHub will scream: **"This branch has conflicts that must be resolved."**

3. The Demonstration: Show how to use VS Code's "Merge Editor" to choose which version of the code to keep.

1. `git remote add upstream https://github.com/MeritoRewak/SBA_RoleA`
2. `git fetch upstream` (just to compare we use fetch – not pull)
3. `git merge upstream/main`
4. Resolve and `git add geometry.py`; `git commit -m „Conflict resolved“`
5. `Git push origin feature-circle-area`

4. Lesson: Git isn't magic—sometimes humans have to decide which code is correct when two people change the same line.



GitHub Action

GitHub Action – CI Experiment Workflow



The goal of this experiment is to demonstrate how **GitHub Actions** automatically validate code through a predefined pipeline. Its purpose is to show how **Continuous Integration (CI)** prevents human error by ensuring that only mathematically correct and stable code is merged into the main project.

Scenario to practice:

1. **Prepare geometry.py:** Write your mathematical functions.
2. **Prepare test_logic.py:** Write a test script using unittest to verify the math results.
3. **Create .github/workflows/main.yml:** Configure the Linux Runner.
4. **Perform a PUSH:** Monitor the "Actions" tab in your browser to watch the Linux VM boot up and execute your tests.
5. **Simulate a Failure:** Intentionally change a formula in your code to be incorrect, push again, and observe how the system automatically rejects the change (Red status).

Step 9: Create the Test Suite (test_logic.py)



Now, create a script that "knows" the correct answers. This script will be used by the GitHub robot to verify your work.

1. Create a file named *test_logic.py* and add the following code:

```
import unittest
from geometry import triangle_area, square_area

class TestGeometry(unittest.TestCase):
    def test_triangle(self):
        # Verification: If base=10 and height=5, area must be 25.0
        self.assertEqual(triangle_area(10, 5), 25.0)

    def test_square(self):
        # Verification: If side=4, area must be 16
        self.assertEqual(square_area(4), 16)

if __name__ == "__main__":
    unittest.main()
```

Step 10: Configure the Automation Pipeline (main.yml) on GitHub



This is where you give instructions to the GitHub Runner (the Linux VM).

1. In your project root, create a folder named `.github`.
2. Inside `.github`, create a folder named `workflows`.
3. Inside `workflows`, create a file named `main.yml`
4. The file structure should be:

```
SBA/ (Project Root)
|— .github/
|   |— workflows/
|       |— main.yml
|— geometry.py
|— test_logic.py
|— .gitignore
```

YAML Syntax (.yaml) – The Configuration Language



YAML (YAML Ain't Markup Language) is a human-friendly data serialization language. Unlike Python, it doesn't "run" logic; it **defines the environment and instructions** for your automation pipeline.

Three Golden Rules

1. Indentation (Hierarchy):

- Rule: Use spaces, never tabs.
- Purpose: Indentation defines the "parent-child" relationship. If a line is indented, it "belongs" to the section above it.

2. Key-Value Pairs (Data):

- Format: key: value (Must have a space after the colon).
- Purpose: These are the settings. The Key is the parameter name; the Value is your specific setting.

3. Lists (Sequences):

- Format: Items start with a hyphen (-).
- Purpose: Used for steps that must happen in a specific order (e.g., first install, then test).

Core Commands & main.yml Structure



Every automation pipeline on GitHub must be located in the specific directory:
.github/workflows/.

Keyword	Functional Description
name:	The name of the workflow displayed in the "Actions" tab.
on:	The Trigger. Defines the event that starts the VM (e.g., push).
jobs:	A group of tasks to be executed on the server.
runs-on:	Specifies the Operating System (e.g., ubuntu-latest).
steps:	A list of specific commands executed sequentially from top to bottom.
Uses:	The Tool/Module. Imports a pre-built Action (e.g., actions/checkout) to perform a complex task without manual scripting.

Step 11: Configure the Automation Pipeline (main.yml) on GitHub



This is where you give instructions to the GitHub Runner (the Linux VM).

1. In your project root, create a folder named `.github`.
2. Inside `.github`, create a folder named `workflows`.
3. Inside `workflows`, create a file named `main.yml` and paste this configuration:

```
name: Engineering Check
on: [push] # The trigger: run every time code is pushed
jobs:
  run-validation:
    runs-on: ubuntu-latest # Requesting a fresh Linux VM from GitHub
    steps:
      - name: Checkout Code
        uses: actions/checkout@v4 # Step 1: Copy code to the VM
      - name: Setup Python
        uses: actions/setup-python@v4 # Step 2: Install Python environment
        with:
          python-version: '3.10'
      - name: Run Tests
        run: python test_logic.py # Step 3: Execute the test script
```

YAML Structure Explanation



Here is the breakdown of main.yml file. This table explains how each line translates into a specific action performed by the GitHub "Robot" (Runner) during your Engineering Check.

YAML Code	Technical Explanation (What it does)
name: Engineering Check	Workflow Identity: Sets the display name that appears in the "Actions" tab on GitHub.
on: [push]	The Trigger: Tells GitHub to start this process automatically every time anyone sends (git push) new code to the repository.
jobs:	Hierarchy Start: Defines a collection of tasks (jobs) that will run.
run-validation:	Job ID: The unique name for this specific set of tasks.
runs-on: ubuntu-latest	Virtual Infrastructure: Requests a clean, hosted Linux (Ubuntu) virtual machine from GitHub's cloud to run your code.
steps:	Sequence: A linear list of commands that the machine will execute one by one.
- name: Checkout Code	Step 1 Label: A human-readable description for the first task.
uses: actions/checkout@v4	Cloning: Downloads your repository files from GitHub onto the virtual machine's hard drive so it can "see" your code.
- name: Setup Python	Step 2 Label: Description for the environment setup.
uses: actions/setup-python@v4	Installation: Installs the Python interpreter on the Linux machine.
with: python-version: '3.10'	Configuration: Specifies exactly which version of Python to use to ensure consistency across the team.
- name: Run Tests	Step 3 Label: Description for the final validation phase.
run: python test_logic.py	Execution: The actual command that triggers your test script. If this returns an error, the whole "Engineering Check" fails (red light).

Step 12: Perform a PUSH and Monitor



Now, send your code to GitHub to trigger the automation.

1. Open your terminal and run:

```
git add .  
git commit -m "Setup CI pipeline for geometry"  
git push origin main
```

2. Go to your repository on GitHub.com.
3. Click the "Actions" tab.
4. You will see a workflow running. Click on it to see the Linux VM logs. If everything is correct, you will see a Green Checkmark.

Step 12: Summary - Environment Provisioning (Infrastructure as Code)



This stage represents the automated setup where GitHub prepares a virtual environment to execute your project.

- **Job Initiation & Workflow Preparation:** The system reserves resources on the Runner (server) and pulls the appropriate system image.
- **Action: actions/checkout@v4**
 - Git "clones" your code from the repository into the virtual machine's memory.
 - Without this step, the "robot" would have no physical access to the geometry.py and test_logic.py files.
- **Action: actions/setup-python@v4**
 - Installation of a specific Python version.
 - We ensure the code runs in an identical environment for every developer, eliminating the "it works on my machine" problem.

Step 12: Summary - Validation & Testing (Continuous Integration)



This is the heart of the process, where the quality of the merged code from Role A and Role B is objectively verified.

- **Execution: `python test_logic.py`**
 - Action: GitHub runs your test script to check for errors in mathematical logic.
 - Result (.. OK): The dots signify successful tests. In your case, it verified both the original functions (square/triangle) and the new `circle_area` function from Role B.
- **Post-job Cleanup:**
 - Secure deletion of your code from the virtual machine and clearing of access tokens.
- **Conclusion (Objective Validation):**
 - The pipeline confirmed that the merge did not break the project.
 - We have mathematical proof that the `circle_area` function works correctly (Result: Ran 2 tests OK).

Step 13: Simulate a Failure (The "Human Error" Test)



This is the most important part—witnessing the system prevent a mistake.

1. Open geometry.py.
2. Break the formula: Change the triangle area to return $0.6 * \text{base} * \text{height}$.
3. Save and push the change:

```
git add geometry.py
git commit -m "Update triangle formula"
git push origin main
```

4. Return to the Actions tab on GitHub. Watch the process turn Red (Failed). Open the logs to see that the test caught the error: $30.0 \neq 25.0$.